



UMassAmherst

Manning College of Information
& Computer Sciences

Programming Methodology
**Lab 2: Type Signatures & Testing with
Jest**

Wednesday September 16, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review recent material
- Work in groups of 2-3 to solve a few exercises
 - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Reminders

- Homework 2 is due next Monday (9/21) at 11:59pm.
 - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
 - Fill out the excusal form on Canvas before the lab.
 - Waking up late or the bus being late are not excusable reasons.
- Submit what you have at the end of lab to Gradescope.
 - Lab submissions are due at 6:00 pm.
 - The submission counts towards your lab grade.

Announcement

- Time and location of final have been updated on SPIRE
- **Monday, December 21st at 3:30 PM in Totman Gym**
- Check SPIRE the day of the exam in case of last minute changes.

Today's Goals

- Practice writing correct type signatures
- Practice writing tests with the Jest framework

Review of Type Signatures

We can infer the types of values based on the operations applied to them.

```
function f(x, y) {  
  return x + (2 * y);  
}
```

The `*` operator performs multiplication on numbers.

- The signature of **f** must be **f(x: number, y: number): number**
- Sometimes shortened to **f: (number, number) => number**

Review of Type Signatures

Sometimes, we have several possibilities.

```
function g(x, y) {  
    return x + y;  
}
```

+ could be addition or
string concatenation

```
g: (number, number) => number  
or
```

```
g: (string, string) => string
```

```
function h(a) {  
    return a.length > 5;  
}
```

strings and arrays both have the
length property

```
h(a: string): boolean  
or
```

```
h<T>(a: T[]): boolean
```

Review of Type Signatures

When a variable has multiple possible types, we can use a **type parameter** like **T**.

- This lets us write **generic functions**.

Type parameters can be used to dictate which types must "agree"

- For example, **filter** works with the following types
 - An array of type **T[]**
 - A callback of type **T => boolean**
 - A return type **T[]**

Anywhere you see **T**, the types must be the same.

Exercise 1: Type Signatures

(a) What are the type signatures of **f** and **g**?

```
const a = [1,2,3,4];  
const b = a.filter(f);  
const c = b.map(g);
```

(b) Initialize variables **a**, **f**, and **g**, such that the code would run without error. The input and output types of **f** should be different from each other.

```
const h = (a, f, g) => a.map(f).filter(g);
```

Remember:

```
map<A,B>(arr: A[], f: (x: A) => B): B[]  
filter<T>(arr: T[], f: (x: T) => boolean): T[]
```

Solution 1: Type Signatures

(a) What are the type signatures of **f** and **g**?

```
const a = [1,2,3,4];  
const b = a.filter(f);  
const c = b.map(g);
```

- **a** has type **number[]**
- Callback to **filter** must accept type of **a**'s elements and return **boolean**
 - **f(x:number): boolean**
- **filter** returns type of array it is called on, so **b** has type **number[]**
- Callback to **map** must accept type of **b**'s elements, but can return any type
 - **g<T>(x:number): T**
- **c** has type **T[]**

Solution 1: Type Signatures

(b) Initialize variables **a**, **f**, and **g**, such that the code would run without error. The input and output types of **f** should be different from each other.

```
const h = (a, f, g) => a.map(f).filter(g);
```

One possible solution is:

```
const a = ["apple", "banana", "cherry"];  
const f = (s: string): number => s.length;  
const g = (n: number): boolean => n > 5;  
// h(a, f, g) == [6, 6]
```

Review : Jest

Jest is the **testing framework** that we use in this course.

```
// each function has a 'describe' block
describe("func", () => {
  // a 'describe' block can have multiple 'it' blocks
  // each 'it' block represents a test case for the function
  it("should do something", () => {
    ...
  });
  ...
});
```

Review : Jest

In addition to global functions, Jest provides robust testing utilities like **expect**.

```
describe("func", () => {  
  it("should do something", () => {  
    ...  
    // expect can be chained with 'matchers'  
    expect(a).toEqual(b);  
    expect(f).toThrow(e);  
  }); // see jestjs.io/docs/expect for more  
  ...  
});
```

Exercise 2: Testing with Jest

You are given a function **rotateRight**`<T>(arr: T[], k: number): T[]` that returns a new array containing the elements of **arr** rotated to the right by **k** steps.

- The function must not mutate the input array.
- **k** must be a non-negative integer. An error is thrown otherwise.
- Examples:
 - **rotateRight**`([1,2,3,4], 1)` returns `[4,1,2,3]`
 - **rotateRight**`([1,2,3,4], 4)` returns `[1,2,3,4]`

Write comprehensive tests for **rotateRight** using Jest.

Solution 2: Testing with Jest

Start with some **basic cases**.

```
it("should rotate right by 1", () => {  
  expect(rotateRight([1, 2, 3, 4], 1)).toEqual([4, 1, 2, 3]);  
});
```

```
it("should rotate right by 2", () => {  
  expect(rotateRight([1, 2, 3, 4], 2)).toEqual([3, 4, 1, 2]);  
});
```

Solution 2: Testing with Jest

Consider when the output array is an **exact copy**.

```
it("should return a copy if k = 0", () => {  
  const a = [1, 2, 3];  
  const b = rotateRight(a, 0);  
  
  expect(b).toEqual([1, 2, 3]);  
  expect(b).not.toBe(a); // should be a new array  
});  
  
it("should return a copy if k equals length", () => {  
  expect(rotateRight([1, 2, 3, 4], 4)).toEqual([1, 2, 3, 4]);  
});
```

Solution 2: Testing with Jest

Test **various cases for k**.

```
it("should behave like k = k % length if k > length", () => {  
  expect(rotateRight([1, 2, 3, 4], 6)).toEqual([3, 4, 1, 2]);  
});
```

```
it("should handle very large k values", () => {  
  expect(rotateRight([1, 2, 3, 4], 100))  
    .toEqual(rotateRight([1, 2, 3, 4], 100 % 4));  
});
```

Solution 2: Testing with Jest

Consider **error cases for k**.

```
it("should throw an error if k is negative", () => {  
  expect(() => rotateRight([1, 2, 3], -1)).toThrow();  
});
```

```
it("should throw an error if k is a float", () => {  
  expect(() => rotateRight([1, 2, 3], 1.5)).toThrow();  
});
```

```
it("should throw an error if k is a negative float", () => {  
  expect(() => rotateRight([1, 2, 3], -1.5)).toThrow();  
});
```

Solution 2: Testing with Jest

Test **various array lengths and types**.

```
it("should return an empty array for empty input array", () => {  
  expect(rotateRight([], 10)).toEqual([]);  
});
```

```
it("should handle single element arrays", () => {  
  expect(rotateRight([42], 5)).toEqual([42]);  
});
```

```
it("should work with non-numeric types", () => {  
  expect(rotateRight(["a", "b", "c"], 1))  
    .toEqual(["c", "a", "b"]);  
});
```

Solution 2: Testing with Jest

Finally, test that the **original array is not mutated**.

```
it("should not mutate the input array", () => {  
  const a = [1, 2, 3, 4];  
  rotateRight(a, 1);  
  
  expect(a).toEqual([1, 2, 3, 4]);  
});
```

Recap

	Type Signatures	Testing with Jest
Purpose	Describe the input and output types of a function	Check whether code behaves correctly
Core idea	Infer types from how values are used	Test expected behavior systematically
Key concepts	Input/Output types, Generics (T) map vs. filter signatures	describe, it, expect matchers
What to check	<i>Do the types agree throughout the expression?</i>	Normal cases, edge cases, errors, mutation
Takeaway	Follow the types through the code	Test more than just the happy path